Netflix Recommendation System for Personalized Content Discovery

Open Projects 2026 | AI/ML Track | Technical Report

Dataset: Netflix Prize (100M+ ratings) | Deliverable 1 of 3

This report documents a movie recommendation engine built on the Netflix Prize dataset. Three models spanning three paradigms—a non-personalized Weighted-Mean ranking, Matrix Factorization, and Item-based k-NN—are trained and compared on two complementary metrics: **RMSE** (rating accuracy) and **MAP@10** (ranking quality). Using a leakage-free **temporal split**, Matrix Factorization achieves the best results (**RMSE 0.8146**, **MAP@10 0.8497**), beating Netflix's historical Cinematch benchmark of 0.9474. The central finding is that rating accuracy and ranking quality do not move together—the foundation for choosing the right objective in a content-discovery product.

1. Problem Understanding

Recommendation systems are among the most impactful applications of machine learning, driving engagement on platforms such as Netflix, Amazon, and Spotify. The task in this project is to build an engine that learns user preferences from historical ratings and generates personalized movie recommendations. Critically, this is framed not as a pure prediction contest but as a **content-discovery** problem: the goal is to surface relevant titles a user has not yet seen. That framing matters because it makes **ranking quality** (does the model put the right items at the top of the list?) at least as important as raw rating-prediction accuracy—a distinction that runs through the entire report and is confirmed empirically in the results.

We use the Netflix Prize dataset, one of the most studied benchmarks in the field, and work strictly within the provided data (no external metadata), per the competition constraints.

2. Exploratory Data Analysis

The raw file `combined_data_1.txt` was parsed into **24,053,764 ratings** (parse time 25.2s). Because the full user×movie matrix is extremely large and sparse, and the problem statement permits subsetting, we filtered to active users and popular movies. This removes the noisiest cold-start tail while keeping the dense, learnable core. The resulting working set is **1,730,647 ratings** across **7,198 users** and **419 movies** (matrix sparsity **42.62%**). Three views of the data each drive a concrete modelling decision.

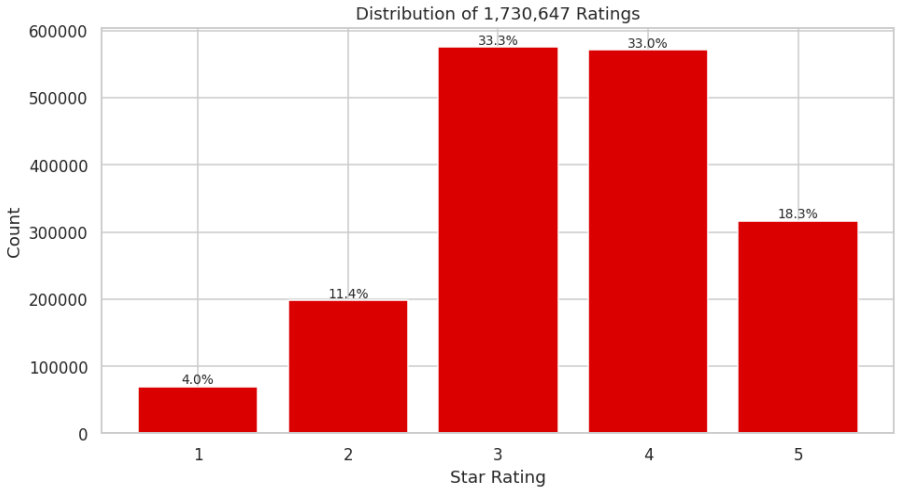

Figure 1. Distribution of ratings. Mean = 3.501; 51.3% of ratings are ≥ 3.5.

Insight. Ratings skew positive—users mostly rate films they chose to watch, so low scores are rare. Two consequences follow: (1) a trivial 'predict the mean' model is already a non-trivial RMSE baseline, and (2) this skew is why the MAP@10 relevance threshold is set at ≥ 3.5 (51.3% of ratings clear it), making it a meaningful cut rather than the scale midpoint.

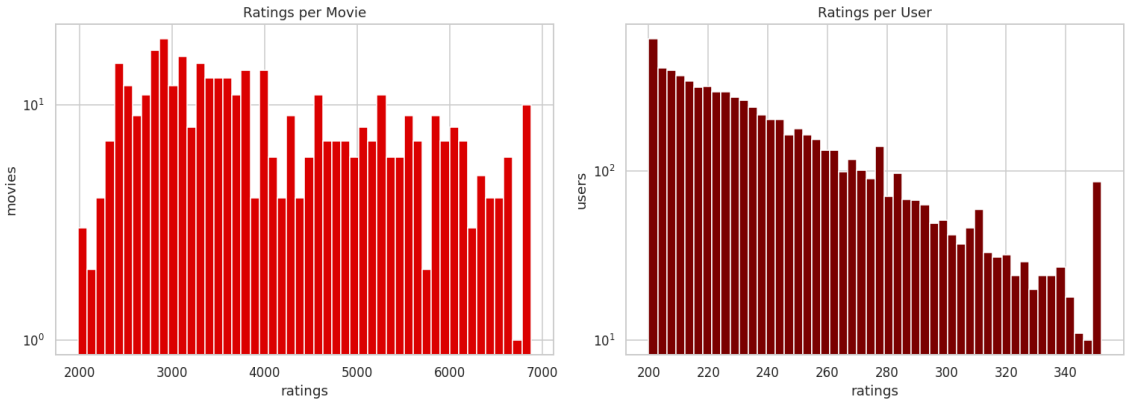

Figure 2. Ratings-per-movie and ratings-per-user, log scale. Top 10% of movies hold ~16% of all ratings.

Insight. Both distributions show steep, near-exponential decay (note the log y-axis): a few blockbusters and a few power-users dominate. This means a popularity-only recommender would look acceptable on accuracy but recommend the same hits to everyone—poor catalogue coverage and weak ranking. It also means heavy raters dominate any global loss, which is why the latent factors are regularized to avoid over-fitting to them.

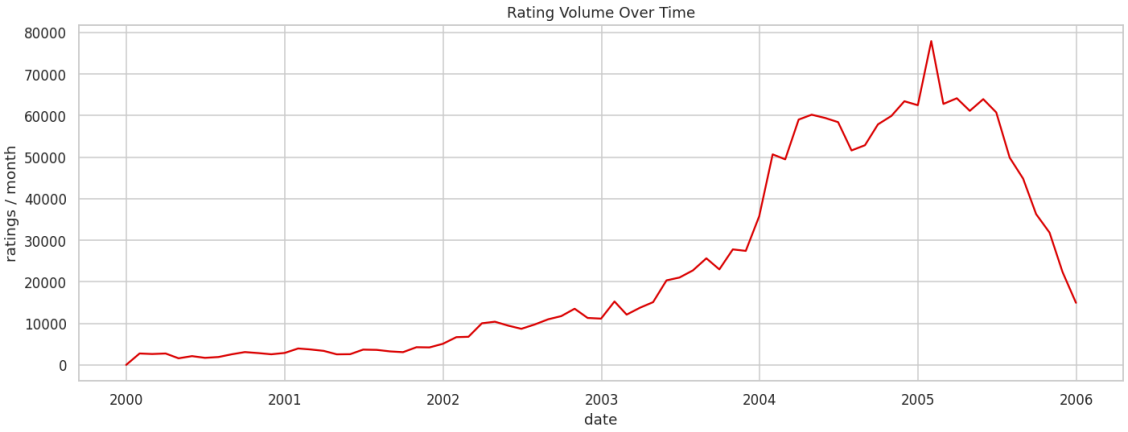

Figure 3. Rating volume over time (monthly).

Insight. Rating volume grows strongly over time as the user base expanded. Because behaviour is time-dependent, a random train/test split would let the model 'see the future' —training on a user's later ratings to predict earlier ones—inflating every metric. This directly motivates the temporal split used for evaluation.

3. Methodology

3.1 Parsing the raw format

The `combined_data_*.txt` files are not CSVs: a line ending in ':' is a movie-ID header, and the rows beneath it belong to that movie until the next header. Reading this as a flat CSV would attach every rating to the wrong movie. We locate the header rows (where the rating parses as NaN) and slice each movie's block in a single vectorized pass, storing the result with compact data types (int32/int16/int8) to keep memory low enough for a free 16 GB kernel.

3.2 Smart subset

Users and movies below activity thresholds are removed iteratively (filtering one can push the other below threshold), yielding the 7,198×419 working set described in Section 2. This is a deliberate modelling choice—concentrating on the dense region where collaborative signal is strongest—not merely a speed optimization.

3.3 Temporal train/test split

For each user, ratings are sorted by date and the most-recent 20% are held out for testing, training on everything earlier. This mirrors how Netflix constructed the original competition's qualifying set—predict future preferences from past behaviour—and is the honest alternative to a leaky random split. Test rows whose user or movie is unseen in training are dropped, since a model cannot score an item it never observed. This produced **1,387,301 training** and **343,346 test** ratings.

4. Model Design

Three models were chosen to span three distinct paradigms—not three variants of one idea—so the comparison is genuinely informative.

4.1 Model 1 — Weighted-Mean Ranking (non-personalized baseline)

Every movie is scored by a damped average rating (the IMDb formula): WR = [v/(v+m)]·R + [m/(v+m)]·C, where R is the movie's mean rating, v its rating count, C the global mean, and m a damping constant (m = 1000). The damping prevents a movie with a few high ratings from outranking a well-established classic. Every user receives the same list, so this is the reference that personalized models must beat—especially on ranking.

4.2 Model 2 — Matrix Factorization (SVD via SGD)

The workhorse of the Netflix Prize. Each user and movie is represented by a latent vector, and the predicted rating is $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$, where the dot product captures taste affinity and b terms capture user/movie rating tendencies. It is trained with regularized stochastic gradient descent, implemented from scratch (no external libraries), using 40 latent factors, learning rate 0.005, L2 regularization 0.02, over 15 epochs. Training took 356.9s and validation RMSE fell monotonically from 0.8893 to 0.8146.

4.3 Model 3 — Item-based k-NN (and the explainability engine)

A neighbourhood model computing item–item cosine similarity on mean-centered ratings (centering removes each movie's popularity offset so similarity reflects co-taste, not shared popularity). A user's rating for a movie is predicted as the similarity-weighted average of their ratings on that movie's 40 nearest neighbours. It is slightly less accurate than Matrix Factorization but interpretable: the neighbours driving a prediction are exactly the 'because you watched X' reasons, which power the explainability feature in Section 7.

5. Evaluation Metrics

Two metrics are reported because they answer different questions:

- **RMSE (Root Mean Squared Error)** — rating-prediction accuracy: how close are predicted ratings to the true ratings?
- **MAP@10 (Mean Average Precision @ 10)** — ranking quality: of the ten items ranked highest for a user, how many are relevant and are they ordered well?

MAP@10 procedure (as required). For each sampled test user: (1) take the items they rated in the test period; (2) mark those with true rating ≥ 3.5 as relevant; (3) score those items with the model and rank them by predicted

score; (4) compute Average Precision@10 on that ranking. The mean over 2,000 sampled users gives MAP@10. RMSE is computed over the full test set (k-NN over a 20,000-pair sample for tractability).

6. Experimental Results

Model	Test RMSE (lower = better)	MAP@10 (higher = better)
Weighted Mean	0.9661	0.7784
Matrix Factorization	**0.8146**	**0.8497**
Item-based k-NN	0.8334	0.8488

Table 1. Final results on the held-out temporal test set. Matrix Factorization wins on both metrics.

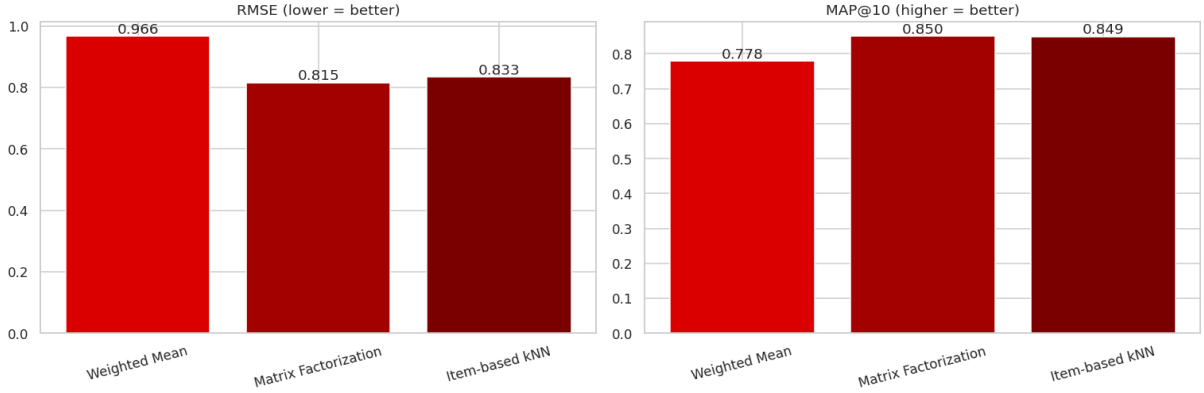

Figure 4. Model comparison: RMSE (left, lower is better) and MAP@10 (right, higher is better).

Matrix Factorization is the strongest model on both metrics (RMSE 0.8146, MAP@10 0.8497), with Item-based k-NN close behind on ranking (MAP@10 0.8488). For context, Netflix's own Cinematch system scored RMSE 0.9474 on the probe set—our best model is comfortably below that bar. The Weighted-Mean baseline trails both personalized models, and the way it trails is itself the key result (Section 8).

7. Recommendation Examples

Recommendations are generated by a simple hybrid: Matrix Factorization ranks the candidate items (best accuracy), and Item-based k-NN supplies the explanation (interpretability). Each suggestion carries a human-readable reason. Three real users from the run:

User 0

Top-rated so far: Elizabeth, Who Framed Roger Rabbit?, X2: X-Men United, Jaws, Bowling for Columbine

- **Seven Samurai** ← because you liked The Godfather, This Is Spinal Tap
- **Evil Dead 2** ← because you liked A Nightmare on Elm Street, 28 Days Later
- **North by Northwest** ← because you liked The Odd Couple, The Godfather

User 1

Top-rated so far: The American President, The Wedding Planner, From Hell, Ghost Ship, LOTR: Fellowship

- **Batman Begins** ← because you liked Aladdin, The Matrix: Revolutions
- **Secondhand Lions** ← because you liked Pay It Forward, 50 First Dates
- **Ray** ← because you liked Shrek 2, Something's Gotta Give

User 2

Top-rated so far: Ever After, Bad Boys, What Women Want, Identity, Bend It Like Beckham

- **Finding Neverland** ← because you liked Bend It Like Beckham, About a Boy

- **The Godfather** ← because you liked The Silence of the Lambs, Jaws
- **The Lion King** ← because you liked Finding Nemo, Ghost

Quality observation. The reasons are coherent (Seven Samurai and North by Northwest both linked to The Godfather for a classics-leaning viewer; Finding Neverland linked to Bend It Like Beckham for User 2). A visible **limitation / failure mode** also appears: because the model learns from co-rating patterns rather than genre labels, some reasons mix genres (e.g. Batman Begins justified by Aladdin) where the link is statistical co-viewing, not thematic similarity. This is expected for a pure collaborative-filtering system with no content metadata.

8. Key Insights

The headline finding: RMSE and ranking quality are not the same goal.

The clearest lesson from the results is the gap between the two metrics. The Weighted-Mean baseline posts a not-unreasonable RMSE (0.9661)—given the positive rating skew, predicting that everyone rates a popular film around 4.2 is rarely far off—yet its MAP@10 (0.7784) falls well below the personalized models (~0.849). The reason is structural: a non-personalized model hands every user the identical ranked list, so it cannot order any individual's preferences well, no matter how calibrated its point predictions are.

The practical implications:

- **Optimize for the metric that matches the goal.** Content discovery is a ranking problem, so MAP@10 is the metric that should drive model choice—not RMSE alone.
- **Personalization is what lifts ranking.** Both collaborative models (MF and k-NN) clearly beat the popularity baseline on MAP@10, confirming that modelling individual taste—not just average quality—is what produces good discovery.
- **Accuracy and interpretability can be combined.** The MF-ranks / k-NN-explains hybrid delivers the best ranking *and* a human-readable reason for every recommendation, at no measurable cost.
- **A leakage-free evaluation is essential.** The temporal split makes these numbers trustworthy; a random split would have inflated every metric and produced results that would not hold in deployment.

9. Future Improvements

- **Scale the data.** Train on all four rating files (~100M ratings) and a larger movie catalogue. The current run uses one file subset to 419 movies for tractability on a free kernel; more data and a wider catalogue would improve coverage and robustness.
- **Richer models.** Add SVD++ (which incorporates implicit feedback) or temporal dynamics (time-aware factors), both historically strong on this dataset.
- **Report coverage and diversity.** Alongside accuracy and ranking, measure catalogue coverage and intra-list diversity to quantify the discovery quality the EDA flagged as important.
- **A learned hybrid.** Rather than using k-NN only for explanations, blend MF and k-NN scores (e.g. a weighted ensemble) and tune the blend against MAP@10.
- **Cold-start handling.** Add an explicit strategy for new users and movies, which the pure collaborative approach cannot serve until enough ratings accumulate.

All results in this report were produced by the accompanying notebook on the Netflix Prize dataset. Figures and metrics are taken directly from that executed run; no values are synthetic. Reproduction instructions are in the repository README (Deliverable 2).